

Лабораторная работа №2

Шигапов Камиль Ильясович

Группа 4404

Задание 4.

Несколько запусков программного кода:

8 потоков -

```
fill_random(A, row_a, col_a, gen, dis);
fill_random(B, row_b, col_b, gen, dis);

// Последовательный
double start_serial = omp_get_wtime();
multiply_serial(A, B, C_serial, row_a, col_a, col_b);
double end_serial = omp_get_wtime();
double time_serial = end_serial - start_serial;

// Параллельный
double start_parallel = omp_get_wtime();
multiply_parallel(A, B, C_parallel, row_a, col_a, col_b, num_threads);
double end_parallel = omp_get_wtime();
double time_parallel = end_parallel - start_parallel;

// Вывод
printf("Serial time: %.4f sec\n", time_serial);
printf("Parallel time: %.4f sec\n", time_parallel);

free_matrix(A, row_a);
free_matrix(B, row_b);
free_matrix(C_serial, row_a);
free_matrix(C_parallel, row_a);

system("pause");
return 0;
```

Matrix A rows: 854
Matrix A cols: 645
Matrix B rows: 645
Matrix B cols: 958
Threads: 8
Serial time: 1.8570 sec
Parallel time: 0.2213 sec
Для продолжения нажмите любую клавишу . . .

4 потока —

```
fill_random(A, row_a, col_a, gen, dis);
fill_random(B, row_b, col_b, gen, dis);

// Последовательный
double start_serial = omp_get_wtime();
multiply_serial(A, B, C_serial, row_a, col_a, col_b);
double end_serial = omp_get_wtime();
double time_serial = end_serial - start_serial;

// Параллельный
double start_parallel = omp_get_wtime();
multiply_parallel(A, B, C_parallel, row_a, col_a, col_b, num_threads);
double end_parallel = omp_get_wtime();
double time_parallel = end_parallel - start_parallel;

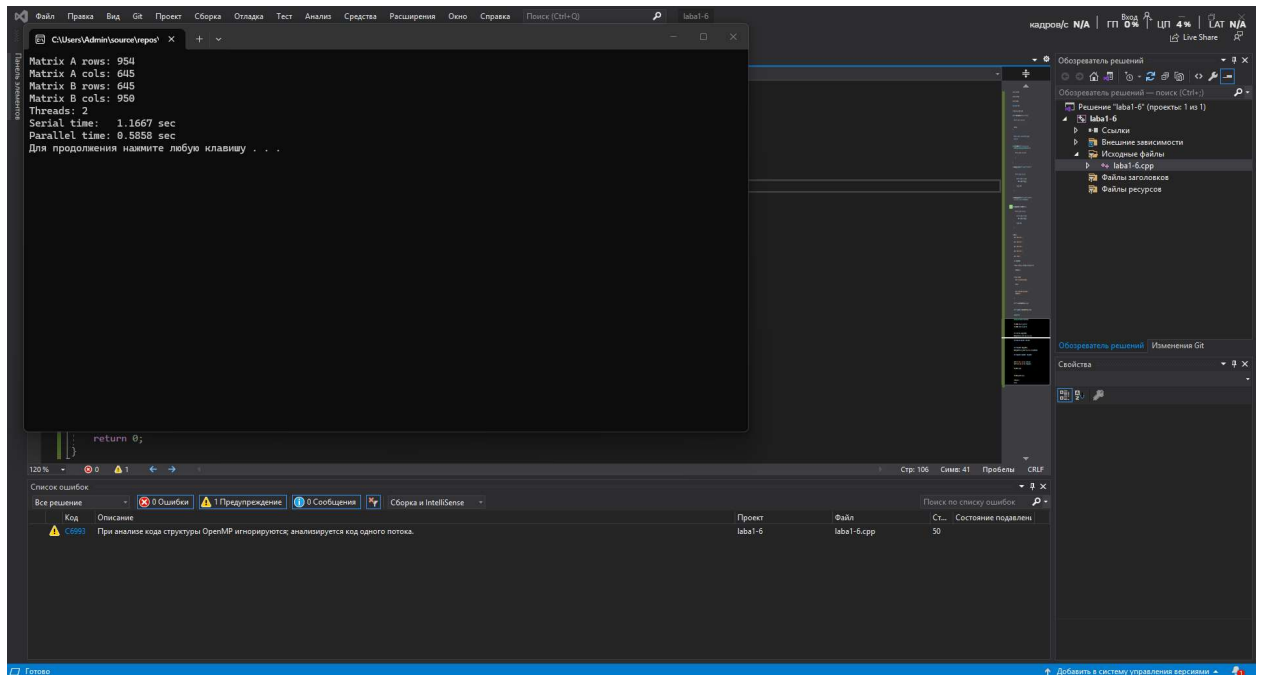
// Вывод
printf("Serial time: %.4f sec\n", time_serial);
printf("Parallel time: %.4f sec\n", time_parallel);

free_matrix(A, row_a);
free_matrix(B, row_b);
free_matrix(C_serial, row_a);
free_matrix(C_parallel, row_a);

system("pause");
return 0;
```

Matrix A rows: 854
Matrix A cols: 645
Matrix B rows: 645
Matrix B cols: 958
Threads: 4
Serial time: 1.8718 sec
Parallel time: 0.2942 sec
Для продолжения нажмите любую клавишу . . .

2 потока —



Параллельная версия оказалась быстрее последовательной более чем в 2 раза уже на 2 потоках, а на 8 потоках ускорение достигло почти 5×.

Это подтверждает, что задача умножения матриц хорошо распараллеливается по следующим причинам:

- Итерации внешнего цикла независимы — вычисление $C[i][j]$ зависит только от строки $A[i][*]$ и столбца $B[*][j]$. Разные потоки работают с разными строками матрицы C , поэтому нет зависимости по данным и нет race condition.
- Большой объём вычислений — сложность алгоритма $O(N^3)$. Для $N = 500$ это ~125 млн операций, что достаточно много, чтобы накладные расходы на создание потоков перекрывались выигрышем от параллелизма.
- Равномерная нагрузка — все итерации внешнего цикла выполняют одинаковое количество операций (N умножений и N сложений). Поэтому `schedule(static)` распределяет работу поровну и не возникает простоя потоков.